In [81]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import zipfile
import cv2
import plotly.express as px


from sklearn.preprocessing import StandardScaler, normalize
from sklearn.cluster import KMeans

%matplotlib inline
```

In [82]:
```
df = pd.read_csv('sales_data_sample.csv', encoding = 'unicode_escape', parse_dates=
df.head()
```

Out[82]:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDER
0	10107	30	95.70	2	2871.00	2003-
1	10121	34	81.35	5	2765.90	2003-
2	10134	41	94.74	2	3884.34	2003-
3	10145	45	83.26	6	3746.70	2003-
4	10159	49	100.00	14	5205.27	2003-

5 rows × 25 columns

In [83]:
```
df.isnull().sum()
```

Out[83]:
```
ORDERNUMBER           0
QUANTITYORDERED       0
PRICEEACH             0
ORDERLINENUMBER       0
SALES                 0
ORDERDATE             0
STATUS                0
QTR_ID                0
MONTH_ID              0
YEAR_ID               0
PRODUCTLINE           0
MSRP                  0
PRODUCTCODE           0
CUSTOMERNAME          0
PHONE                 0
ADDRESSLINE1          0
ADDRESSLINE2       2521
CITY                  0
STATE              1486
POSTALCODE           76
COUNTRY               0
TERRITORY          1074
CONTACTLASTNAME       0
CONTACTFIRSTNAME      0
DEALSIZE              0
dtype: int64
```

In [49]:
```
df_drop  = ['ADDRESSLINE1', 'ADDRESSLINE2', 'POSTALCODE', 'CITY', 'TERRITORY', 'PHO
df = df.drop(df_drop, axis=1)
df.head(3)
```

Out[49]:

	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	STATUS	C
0	30	95.70	2	2871.00	2003-02-24	Shipped	
1	34	81.35	5	2765.90	2003-05-07	Shipped	
2	41	94.74	2	3884.34	2003-07-01	Shipped	

Drop georaphic features and names, phone

In [50]:
```
df.shape
```

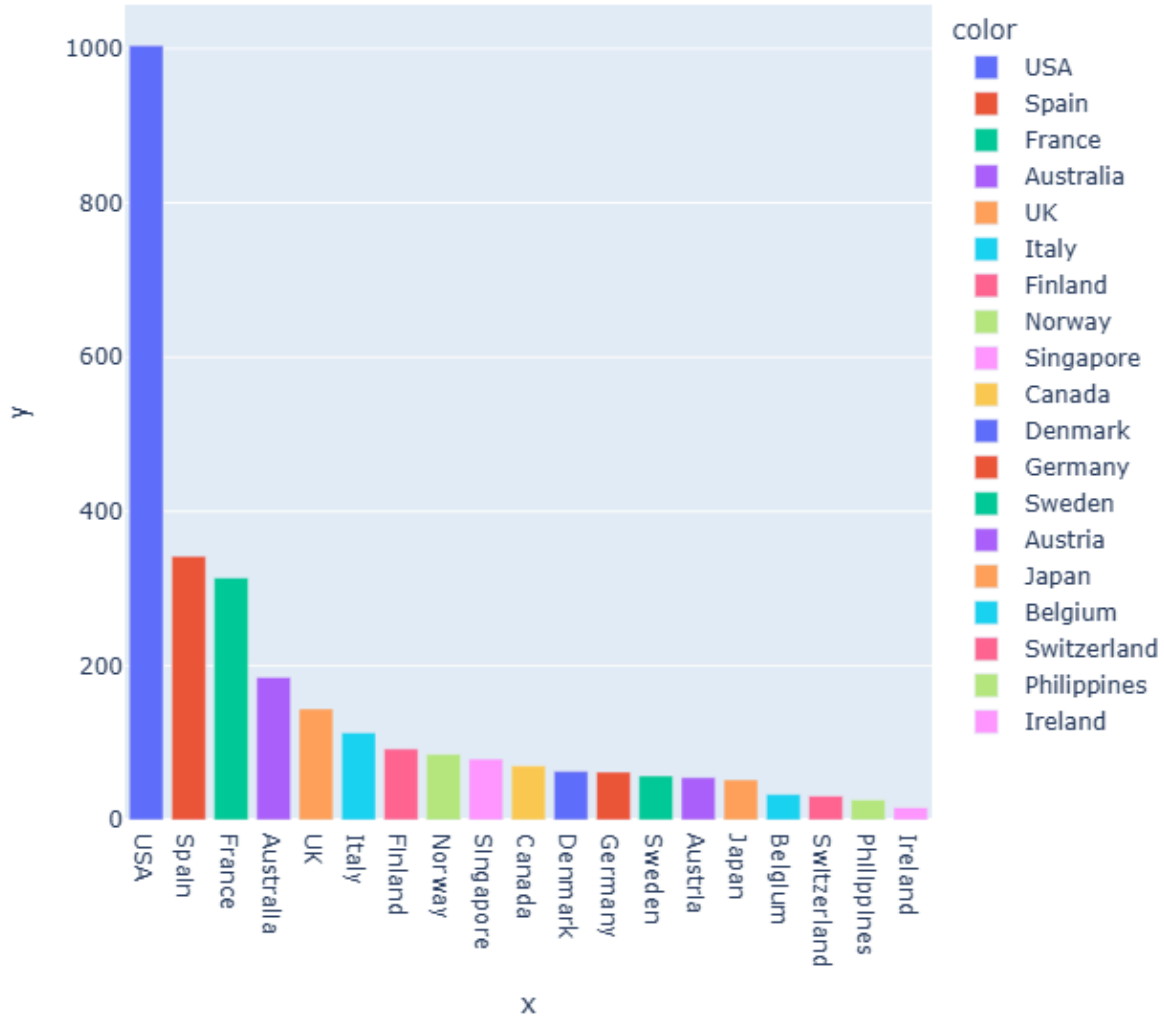
Out[50]:
```
(2823, 14)
```

In [51]:
```
df.isna().sum()
```

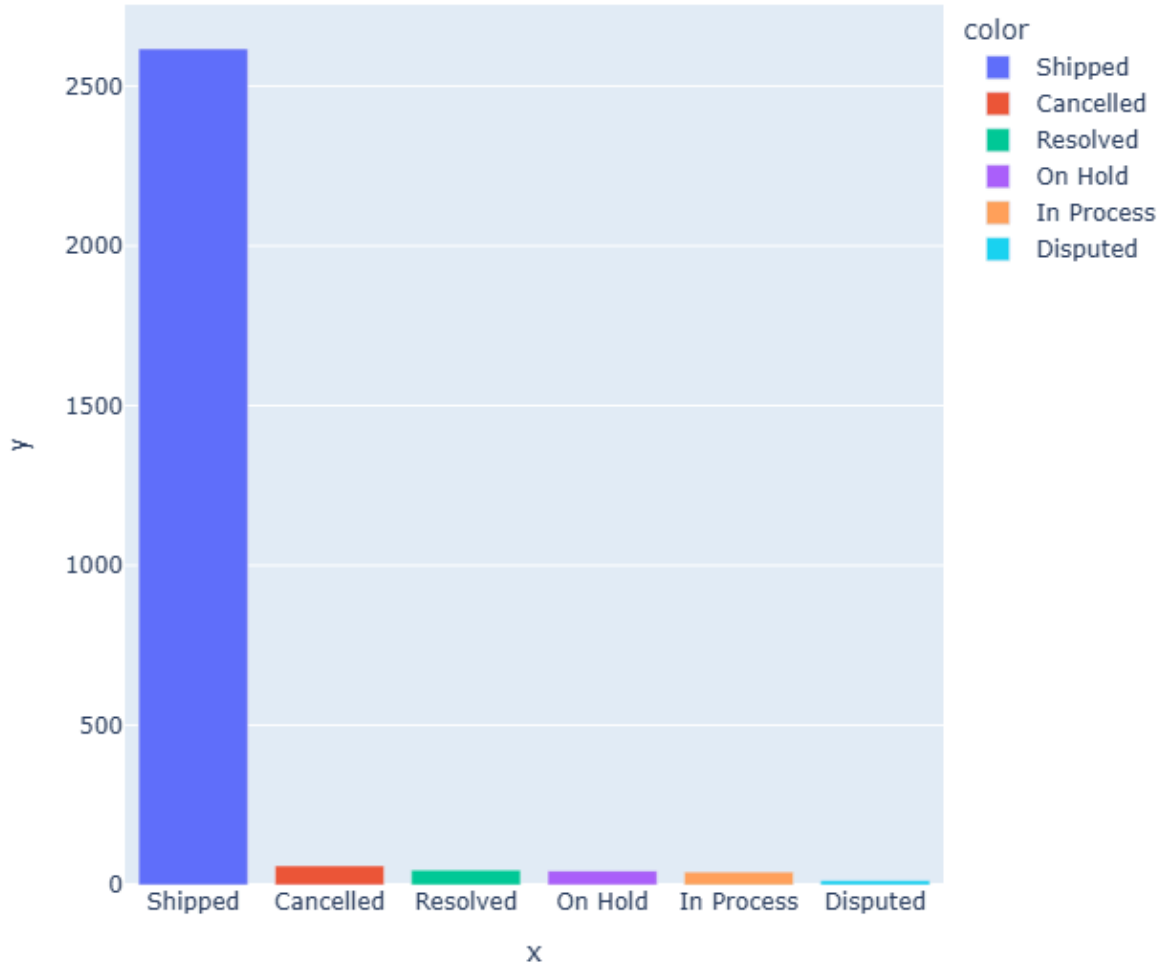
Out[51]:
```
QUANTITYORDERED    0
PRICEEACH          0
ORDERLINENUMBER    0
SALES              0
ORDERDATE          0
STATUS             0
QTR_ID             0
MONTH_ID           0
YEAR_ID            0
PRODUCTLINE        0
MSRP               0
PRODUCTCODE        0
COUNTRY            0
DEALSIZE           0
dtype: int64
```

In [84]:
```
def barplot_visualization(x):
  fig = px.bar(x = df[x].value_counts().index, y = df[x].value_counts(), color = df
  fig.show();
```

In [85]:
```
barplot_visualization('COUNTRY')
```



In [86]:
```
barplot_visualization('STATUS');
```



Drop unbalanced feature

In [55]:
```
df.drop(columns=['STATUS'], axis=1, inplace=True)
```

In [56]:
```
print('Columns resume: ', df.shape[1])
```

```
Columns resume:  13
```

In [57]:
```
barplot_visualization('PRODUCTLINE')
```

In [58]:
```
barplot_visualization('DEALSIZE')
```

Prepare data

In [59]:
```
def dummies(x):
  dummy = pd.get_dummies(df[x])
  df.drop(columns=x, inplace=True)
  return pd.concat([df, dummy], axis = 1)
```

In [60]:
```
df =  dummies('COUNTRY')
```

In [61]:
```
df =  dummies('PRODUCTLINE')
```

In [62]:
```
df =  dummies('DEALSIZE')
```

In [63]:
```
df.head()
```

Out[63]:

	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	QTR_ID	M
0	30	95.70	2	2871.00	2003-02-24	1	
1	34	81.35	5	2765.90	2003-05-07	2	
2	41	94.74	2	3884.34	2003-07-01	3	
3	45	83.26	6	3746.70	2003-08-25	3	
4	49	100.00	14	5205.27	2003-10-10	4	

5 rows × 39 columns

In [64]:
```
y = pd.Categorical(df['PRODUCTCODE'])
y
```

Out[64]:
```
['S10_1678', 'S10_1678', 'S10_1678', 'S10_1678', 'S10_1678', ..., 'S72_3212', 'S72
_3212', 'S72_3212', 'S72_3212', 'S72_3212']
Length: 2823
Categories (109, object): ['S10_1678', 'S10_1949', 'S10_2016', 'S10_4698', ..., 'S
700_3962', 'S700_4002', 'S72_1253', 'S72_3212']
```

In [65]:
```
df['PRODUCTCODE'] = pd.Categorical(df['PRODUCTCODE']).codes
```

In [66]:
```
df.head()
```

Out[66]:

	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	QTR_ID	M
0	30	95.70	2	2871.00	2003-02-24	1	
1	34	81.35	5	2765.90	2003-05-07	2	
2	41	94.74	2	3884.34	2003-07-01	3	
3	45	83.26	6	3746.70	2003-08-25	3	
4	49	100.00	14	5205.27	2003-10-10	4	

5 rows × 39 columns

In [67]:
```
df_group = df.groupby(by='ORDERDATE').sum()
fig = px.line(x = df_group.index, y = df_group.SALES, title='sales_peak')
fig.show();
```

In [68]:
```
df.drop('ORDERDATE', axis=1, inplace=True)
```

drop 'ORDERDATE', 'QTR_ID' because we have 'MONTH' etc.

In [69]:
```
df.drop('QTR_ID', axis=1, inplace=True)
```

In [70]:
```
df.shape
```

Out[70]: (2823, 37)

Use K-MEANS algorithm

In [71]:
```
scaler =  StandardScaler()
df_scaled = scaler.fit_transform(df)
```

In [72]:
```
scores = []
range_values = range(1, 15)
for i in range_values:
  kmeans = KMeans(n_clusters = i)
  kmeans.fit(df_scaled)
  scores.append(kmeans.inertia_)
```

In [73]:
```
plt.plot(scores, 'bx-')
plt.title('Finding right number of clusters')
plt.xlabel('Clusters')
plt.ylabel('scores')
plt.show();
```



The elbow method

In [74]:
```
kmeans = KMeans(4)
kmeans.fit(df_scaled)
```

Out[74]:
```
KMeans
▶ Parameters
```

In [75]:
```
labels = kmeans.labels_
labels
```

Out[75]:
```
array([2, 2, 2, ..., 1, 3, 1], shape=(2823,), dtype=int32)
```

In [76]:
```
kmeans.cluster_centers_.shape
```

Out[76]:
```
(4, 37)
```

In [77]:
```
cluster_centers = pd.DataFrame(data = kmeans.cluster_centers_, columns= [df.columns
cluster_centers
```

Out[77]:

	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	MONTH_ID	YEAR_ID
0	0.191867	-0.057456	-0.010199	0.033722	-0.277772	-0.945247
1	0.385790	0.603984	-0.008400	0.674782	0.001741	-0.000277
2	0.006683	-0.023524	-0.152101	-0.019277	-0.048153	0.056321
3	-0.474310	-0.724427	0.054369	-0.813644	0.018260	0.006047

4 rows × 37 columns

Invert the data

In [78]:
```
cluster_centers = scaler.inverse_transform(cluster_centers)
cluster_centers = pd.DataFrame(data=cluster_centers, columns=[df.columns])
cluster_centers
```

Out[78]:

	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	MONTH_ID	YEA
0	36.961538	82.499615	6.423077	3615.989615	6.076923	2003.15
1	38.850295	95.841320	6.430678	4796.525531	7.098820	2003.81
2	35.157895	83.184056	5.823529	3518.390279	6.916409	2003.85
3	30.473166	69.046333	6.695886	2055.531699	7.159213	2003.81

4 rows × 37 columns

In [79]:
```
sales_of_cluster = pd.concat([df, pd.DataFrame({'cluster': labels})], axis=1)
sales_of_cluster.head()
```

Out[79]:

	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	MONTH_ID	YEAR_ID	M
0	30	95.70	2	2871.00	2	2003	
1	34	81.35	5	2765.90	5	2003	
2	41	94.74	2	3884.34	7	2003	
3	45	83.26	6	3746.70	8	2003	
4	49	100.00	14	5205.27	10	2003	

5 rows × 38 columns